Ricochet

Système de recommandation d'articles — MVP

Approches testées · Fonctionnalités · Architecture cible · Démonstration

Contexte

Ce qui est demandé, avec quelles données

- Proposer **5 articles** à chaque lecteur, pour l'inciter à lire davantage.
- Pas de données propriétaires : jeu public Globo.com — **2 988 181** clics, **322 897** lecteurs, **364 047** articles.

- **Contrainte majeure : le jeu est anonymisé.**
  - aucun titre, aucun texte : un article n'est qu'un vecteur de 250 dimensions et un identifiant ;
  - aucune note, aucun avis : seulement des clics, donc un signal implicite ;
  - conséquence — tout se fait sans lire le contenu, et l'interface ne peut afficher que des identifiants.

Description fonctionnelle

Ce que le service fait, du point de vue de l'utilisateur

Le service — pour un lecteur donné, il renvoie les **N articles** les plus susceptibles de l'intéresser.

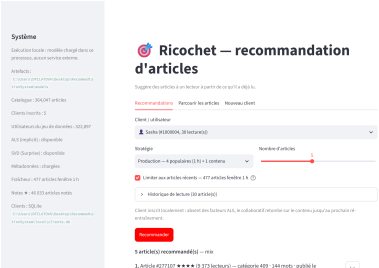
Entrées `user_id` requis
`n` défaut 5 · `method` défaut mixte
`region` · `history` · `fresh_only` facultatifs

Sortie N identifiants d'articles, classés

- **Cinq stratégies** au choix : popularité, contenu, collaboratif ALS, SVD, hybride — la **mixte** par défaut.
- **Lecteur nouveau ou sans historique exploitable** : articles populaires, **adaptés à sa région** si elle est connue.
- **Priorité au contenu récent** : la fenêtre de recommandation est recalculée périodiquement pour tenir compte des articles publiés depuis.
- **Jamais de liste vide**, quelle que soit la requête.

L'application qui l'appelle permet de :

- choisir un lecteur et obtenir ses articles ;
- **marquer un article comme lu** — le profil s'enrichit, la liste suivante change, **sans ré-entraînement** ;
- parcourir le catalogue (364 047 articles) ;
- **inscrire** un nouveau lecteur, recommandé immédiatement.



Partie 1 — Les cinq approches testées

10 minutes · comment chacune classe les articles

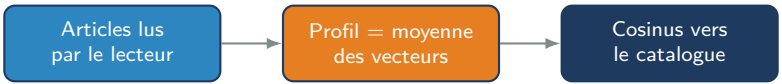
Approche	Principe	Bibliothèque
Popularité	articles les plus lus sur une fenêtre	—
Contenu	profil = moyenne des embeddings, cosinus	scikit-learn (ACP)
Collaboratif ALS	factorisation lecteur × article	implicit
Collaboratif SVD	prédiction de notes dérivées des clics	Surprise
Mixte	combinaison sur les 5 places	—

Les trois approches **personnalisées** sont détaillées dans les trois diapositives suivantes. La **popularité** traverse tous les résultats — c'est elle que mesure « la fraîcheur pèse plus que le modèle ». La stratégie **mixte**, retenue en production, clôt cette partie.

La popularité sert de **référence à battre** : elle ne personnalise rien, donc toute approche plus complexe doit faire mieux qu'elle.

Filtrage par le contenu

Similarité des embeddings d'articles



Avantages

- fonctionne dès le 1er clic, sans ré-entraînement ;
- gère un article nouveau immédiatement (il a un vecteur) ;
- couverture du catalogue la plus large : **0,27 %** contre 0,003 % ;
- personnalisation **96 %**.

Inconvénients

- précision faible : HitRate@5 = **0,0200** ;
- n'exploite pas l'intelligence collective ;
- sans restriction aux articles récents, propose des articles vieux de plusieurs années que personne ne lit.

Collaboratif ALS (`implicit`)

Factorisation pour feedback implicite

Toutes les cases vides sont traitées comme des **négatifs pondérés** : une formulation adaptée à des clics, pas à des notes.

Avantages

- capte des goûts **latents** qu'aucune similarité de contenu ne voit ;
- effet de surprise : recommande hors du thème déjà lu ;
- personnalisation **95 %**.

Inconvénients

- HitRate@5 sur test = **0,0415** ;
- **cold start** : inutile pour un lecteur ou un article nouveau ;
- limite structurelle — un article publié il y a deux heures n'a presque aucune co-lecture, le collaboratif ne peut pas le proposer ;
- réglages sensibles et **couplés** entre eux.

Balayés **séparément** sur validation, les deux gagnants étaient 72 h et 16 facteurs. Croisés, ils donnent **0,0110** — la pire des quatre combinaisons ; 24 h + 16 facteurs donne **0,0345**. On ne juxtapose pas des optima partiels.

Collaboratif SVD (`Surprise`)

Trois définitions de note, trois résultats

Définition de la note	HitRate@5	Person.	Verdict
Étoiles de l'article (log des clics)	0,0015	23 %	ne classe pas
Binaire + 1 négatif échantillonné	0,0080	46 %	insuffisant
Binaire + 4 négatifs échantillonnés	0,0815 *	34 %	meilleure variante

Avantages

- bien posée — binaire + 4 négatifs — c'est la meilleure des trois variantes sur validation ;
- implémentation directe, entraînement rapide, peu de réglages.

Inconvénients

- **ne se reproduit pas** : 0,0815 en validation, mais **0,0220** sur test — un facteur 3,7 ;
- les étoiles ne classent pas du tout (0,0015) : la note ne dépend que de l'article, le modèle apprend « quels articles sont lus », pas « qui lit quoi » ;
- même cold start que l'ALS ; personnalisation 34 % seulement.

Même bibliothèque, même algorithme : c'est la **formulation du problème** qui change tout, pas l'outil. Et c'est une période de test intacte qui révèle qu'un réglage ne s'est pas reproduit.

Comment nous avons su que les premiers chiffres étaient faux

Découpage temporel 60 / 20 / 20

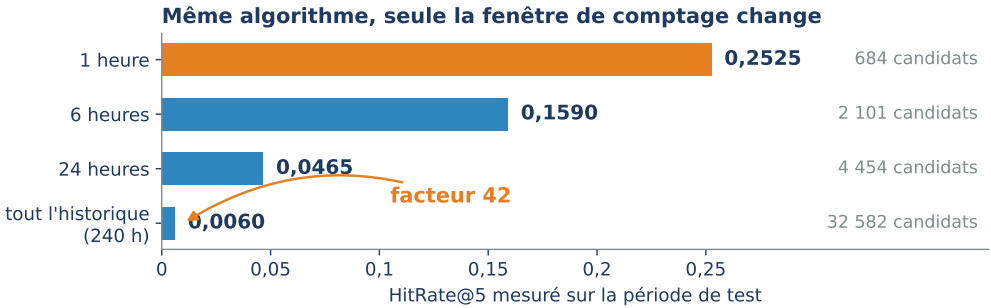
- Première évaluation : ALS entraîné sur 100 % des clics, puis mesuré en retirant le dernier clic de chaque lecteur — un clic que le modèle avait déjà vu. HitRate@5 = **0,2415**.
- Avec un découpage temporel correct, il tombe à **0,0415** — **un facteur 5,8** de précision imaginaire, invisible sans découpage.
- La popularité, elle, ne profite pas de la fuite : son classement ne dépend pas du lecteur. Seuls les modèles *personnalisés* peuvent recopier ce qu'ils ont appris d'un lecteur.
- Temporel et non aléatoire : sur un flux d'actualité, un découpage aléatoire laisse le modèle apprendre des clics *postérieurs* à ceux qu'il doit prédire.
- Le réglage se fait sur la validation ; le test ne sert qu'à la mesure finale, une seule fois.



le temps s'écoule de gauche à droite — aucune fuite du futur vers le passé

Résultat principal : la fraîcheur pèse plus que le modèle

Le résultat le plus important de l'évaluation



- **Facteur 42** entre la première et la dernière barre, sans changer une ligne d'algorithme.
- Explication : **48 %** des articles lus pendant la période évaluée n'existaient pas encore pendant l'entraînement.
- Conséquence produit : recalculer la fenêtre en continu compte davantage que raffiner le modèle.

Quatre critères, pas un seul

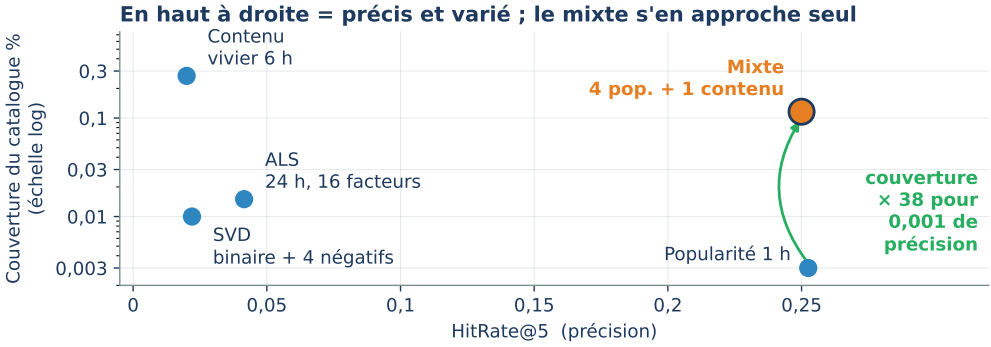
La précision seule désigne toujours la popularité

Critère	Ce qu'il mesure	Popularité	Contenu
HitRate@5	un article recommandé a été lu	0,2525	0,0200
Couverture	part du catalogue exposée	0,003 %	0,27 %
Personnalisation	différence entre deux listes	4 %	96 %
Recall@5	part du futur devinée (plafond 0,76)	0,0555	0,0034

- La popularité est 12 fois plus précise, mais expose **11 articles sur 364 047** : un éditeur y perd son catalogue.
- Aucun critère ne sauve le **SVD sur étoiles** : HitRate nul et personnalisation 12 %, contre 96 % pour le contenu. Il ne trouve rien *et* ne distingue presque pas les lecteurs — parce que sa note ne dépend que de l'article.

Résultat final sur la période de test

Chaque approche dans sa meilleure configuration



- Retenu : **4 populaires (1 h) + 1 contenu** — la place donnée au contenu ne coûte rien en précision (écart non significatif) et **multiplie la couverture par 38**.
- Les deux collaboratifs restent en bas à gauche : un ordre de grandeur moins précis que la popularité *et* vingt fois moins couvrants que le contenu. Disponibles dans l'application, ils ne sont pas servis.

Le système de recommandation retenu

Ce qui tourne en production : la stratégie « mixte »



4 places — les plus lus de la **dernière heure**

1 place — contenu, vivier 6 h

Pourquoi ce partage

- la place donnée au contenu ne coûte pas de justesse mesurable ;
- elle multiplie par **38** la part de catalogue exposée ;
- meilleur compromis justesse / couverture des cinq approches.

Jamais de liste vide

- cascade de repli à quatre niveaux : région ∩ fraîcheur, puis fraîcheur, puis région, puis tout l'historique ;
- `history` transmis dans la requête : un lecteur inconnu du service est tout de même personnalisé.

Partie 2 — Les stratégies dans l'application

6 minutes · le même lecteur, une seule case décochée

Fraîcheur activée — fenêtre 1 h

5 article(s) recommandé(s) — mix

1. Article #277107 ★★★★ (9 373 lecteurs) — catégorie 409 · 144 mots · publié le 16/10/2017

2. Article #336254 ★★★★★ (10 627 lecteurs) — catégorie 437 · 170 mots · publié le 16/10/2017

3. Article #157478 ★★★★ (5 888 lecteurs) — catégorie 281 · 259 mots · publié le 16/10/2017

4. Article #352979 ★★★★ (3 784 lecteurs) — catégorie 442 · 169 mots · publié le 16/10/2017

5. Article #284220 ★ (9 lecteurs) — catégorie 412 · 224 mots · publié le 17/10/2017

Fraîcheur désactivée — tout le catalogue

5 article(s) recommandé(s) — mix

1. Article #4907 ★★★ (603 lecteurs) — catégorie 2 · 210 mots · publié le 09/10/2017

2. Article #257291 ★★★★ (8 119 lecteurs) — catégorie 391 · 205 mots · publié le 04/10/2017

3. Article #2303 ★★★ (111 lecteurs) — catégorie 1 · 204 mots · publié le 02/10/2017

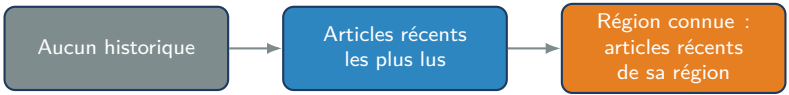
4. Article #235230 ★★★★ (9 123 lecteurs) — catégorie 375 · 262 mots · publié le 03/10/2017

5. Article #160997 ★ (1 lecteurs) — catégorie 281 · 241 mots · publié le 25/09/2017

- À gauche, le 16/10 et 3 784 à 10 627 lecteurs ; à droite, le 25/09 et un article lu par **une** personne — **le facteur 42, rendu visible**.
- Cinquième place à gauche : #284220 du **17/10**, neuf lecteurs. C'est la place du contenu, et la popularité ne la trouverait jamais.

Un lecteur dont on ne sait rien

Le cas le plus fréquent sur un portail d'actualité

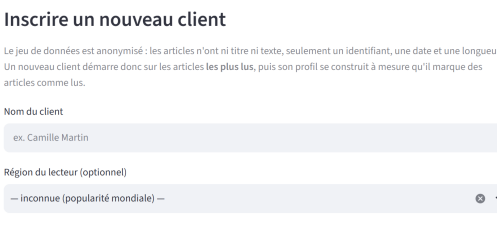


- **Cascade de repli** : région croisée avec la fraîcheur, puis fraîcheur, puis région, puis historique complet. Aucune requête ne renvoie de liste vide.
- La région est la seule information exploitable sur un lecteur inconnu : 27 régions retenues, celles ayant assez de clics pour un classement.
- Dès le premier article lu, le profil de contenu prend le relais — **sans aucun ré-entraînement**.

Inscrire un lecteur, voir son profil se former

En direct, sans ré-entraînement

Inscription — région optionnelle



Catalogue — 364 047 articles



- Recommandations **immédiates** à l'inscription ; à chaque article lu, le profil — moyenne des vecteurs lus — est recalculé à la demande, **sans ré-entraînement**.
- Le catalogue a une raison d'être : sans lui, le lecteur ne cliquerait que ce que le modèle propose déjà, et son profil ne ferait que se renforcer.
- L'historique est **transmis dans la requête** : la Function recommande un lecteur qu'elle ne connaît pas, sans état.

Publier un nouvel article

Disponible en minutes, sans ré-entraînement



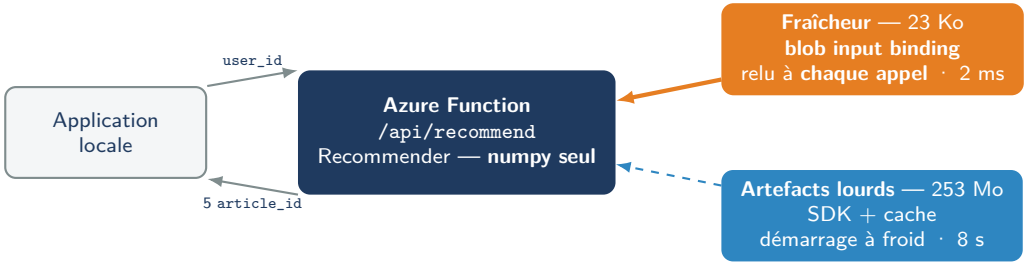
- La projection ACP est sérialisée avec le catalogue (**51 Ko**) : sans elle, intégrer un article imposerait de réajuster l'ACP, donc de recalculer les 364 047 vecteurs existants.
- Vérifié : trois articles ajoutés reçoivent les identifiants 364047–364049 et **entrent immédiatement** dans le top-5 d'un lecteur proche.
- L'article n'a ni étoiles ni facteurs collaboratifs — il n'a aucun clic. Il est recommandé par le contenu, seul signal disponible à cet instant.

Élément	Rôle
Découpage temporel 60/20/20	réglage sur validation, mesure sur test
MLflow (SQLite)	paramètres, métriques et commit git de chaque essai
Registre de modèles	versions des artefacts, retour arrière possible
Porte de qualité	bloque la publication si HitRate@5 baisse de >10 %
CI (GitHub Actions)	87 tests, copies déployées, secrets, Terraform

- La porte compare aux métriques de référence **versionnées dans le dépôt** ; la référence ne change que délibérément.
- Sans elle, un ré-entraînement dégradé se publierait sans alerte — c'est exactement ce qui est arrivé avec un chargeur d'artefacts incomplet, qui servait la popularité de tout l'historique **sans lever d'erreur**.

Partie 3 — Architecture retenue : l'architecture 2

Serverless *sans* API dédiée — la Function lit Blob Storage elle-même



- **Sans état.** La Function ne conserve rien entre deux requêtes : le profil d'un lecteur qu'elle ne connaît pas arrive dans le paramètre `history` de l'appel.
- **Mais pas sans mémoire.** L'instance `Recommender` est un **singleton** réutilisé d'une invocation à l'autre : les 253 Mo sont lus une seule fois par instance, au démarrage à froid, après **découverte dynamique** du conteneur — le chargeur énumère les blobs, il ne suit pas une liste figée.
- **Deux accès, pas un.** Le *binding* relit son blob à chaque invocation : ce qu'il faut pour 23 Ko qui changent toutes les heures. Appliqué aux 253 Mo il supprimerait le cache — **8 s par appel au lieu de 0,2 s**, 40 fois plus.

Ce qui sépare un service d'une démonstration

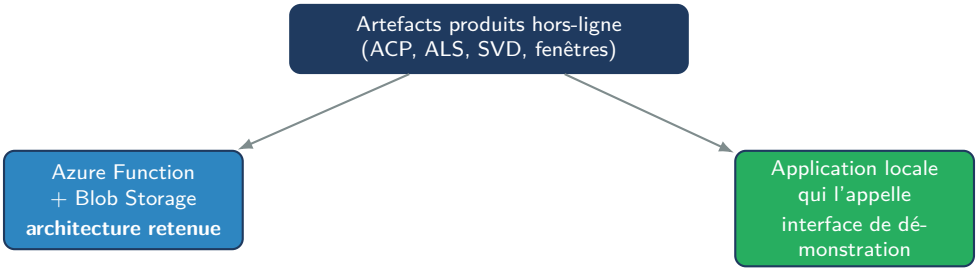
Quatre défaillances silencieuses, et le garde-fou de chacune

Défaillance	Garde-fou	Preuve
Dérive d'artefact observée en production	validité du cache jugée sur la taille **et** la date du blob — un modèle reconstruit de taille identique est retéléchargé	`test_meme_taille_mais_blob_plus_recent`
Cache périmé observée en production	le conteneur est **énuméré** à chaque démarrage ; une liste figée avait fait manquer les artefacts ajoutés après elle	`list_blobs()`, trois artefacts requis contrôlés
Catalogue incohérent	tout identifiant hors catalogue est filtré, au chargement *et* à chaque requête — sinon l'indexation échoue	`test_historique_hors_catalogue_filtre`
Interférence entre requêtes concurrentes	la surcharge `history` vit dans un `threading.local`, remise à zéro en `finally` : le profil d'un appel ne fuit pas vers un autre	isolation par requête, `src/recommender.py`

Aucune de ces quatre défaillances ne lève d'erreur. Le service continue de répondre 200, avec de mauvaises recommandations — les deux premières ont été découvertes en comparant la réponse déployée à la réponse locale, pas par une exception.

Le service et l'application qui l'appelle

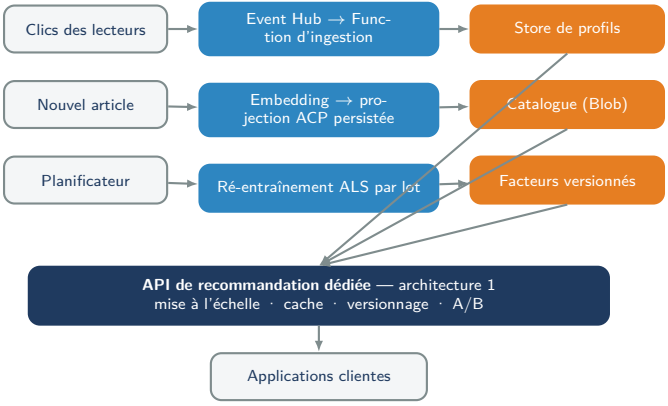
2 minutes · une seule source de vérité, des copies générées



- Les mêmes artefacts hors-ligne alimentent le service et l'application : les copies déployées du cœur sont **générées** depuis `src/`, et la CI vérifie qu'elles sont à jour.
- À l'inférence, aucune bibliothèque d'apprentissage : **numpy seul**. scikit-learn, implicit et Surprise ne servent qu'hors-ligne.
- L'application ne calcule rien : elle appelle l'endpoint et affiche les cinq identifiants — c'est bien **la Function** qui recommande.

Architecture cible

Absorber les nouveaux articles et les nouveaux lecteurs



Seule la ligne du bas impose un ré-entraînement. Les deux besoins immédiats — un article visible tout de suite, un lecteur servi dès son premier clic — passent par les deux premières lignes, sans ré-apprentissage.

Architecture cible — ce que coûte chaque nouveauté

Latence atteignable et besoin de ré-entraînement

Événement	Traitement	Latence	Ré-entr.
Nouvel article	embedding + projection ACP	minutes	non
Nouveau clic	profil recalculé à la demande	temps réel	non
Nouveau lecteur	popularité, puis contenu au 1er clic	temps réel	non
Couverture collab.	facteurs latents	heures	oui
Dérive du catalogue	ré-ajustement de l'ACP	planifié	oui

- **Seule la partie collaborative impose un ré-entraînement** ; les deux besoins immédiats sont couverts sans.
- Reste à construire : ingestion événementielle, store de profils, fenêtre glissante recalculée en continu, index de similarité pour l'échelle.

Partie 4 — Démonstration

2 minutes · six gestes

1. Sélectionner un lecteur : 5 articles, avec étoiles et nombre de lecteurs.
2. Décocher « articles récents » : la liste change — **l'effet mesuré, en direct**.
3. Basculer sur « SVD Surprise » : autre classement, autre bibliothèque.
4. Parcourir le catalogue : cliquer un article hors des suggestions.
5. Inscrire un lecteur : il reçoit des recommandations immédiatement.
6. Le même appel sur Azure : 0,2 s, réponse identique au local.

Les articles n'ont ni titre ni texte : le jeu de données est anonymisé par sa source. L'interface affiche identifiant, catégorie, longueur, date et note — les seules informations réelles disponibles.

La fraîcheur pèse plus que le modèle

Facteur 42 sur la précision, mesuré sur la période de test.

Une place sur cinq accordée à la personnalisation ne coûte rien et multiplie par 38 la part du catalogue exposée.